

TokTickIT

Requester-facing ticket workflow

Student	GUNTEE DOUNGMANEE
Repository	ovenmakemeheat/tokticket
Report Issue	Issue #65 - Lab 2 final submission report
Integrated baseline	lab2-staging at 83da75e
Stable branch	main at b9903e8 (Lab 1 release)
Stack	React, TypeScript, Vite, Bootstrap, Express, Prisma, PostgreSQL

DRAFT - 2026-09-04. Lab 2 implementation and recorded evidence are integrated on **lab2-staging**. Eight additional report-state captures are now recorded. The release PR to final **main**, final-main test output, human visual sign-off, and several submission captures are still pending.

Contents

1	Answer Part 1: Git Use with Engineering Workflow	2
2	Answer Part 2: Spec DD	3
3	Answer Part 3: Test DD and Traceability	3
4	Answer Part 4: AI Use with Reflection	4
5	Answer Part 5: Development Requester Select Screen	5
6	Answer Part 6: Working Ticket Screen - Create Mode	5
7	Answer Part 7: Working My Tickets Screen	6
8	Answer Part 8: Working Ticket Screen - View Mode and Attachments	7
9	Answer Part 9: Zen Green UI and Responsive Evidence	8

1 Answer Part 1: Git Use with Engineering Workflow

Lab 2 uses one feature branch per Issue and integrates each reviewed Pull Request into `lab2-staging`. The human reviewer merges the feature PR; the PR author does not merge it. The current repository state is:

```
main (b9903e8, Lab 1 release)
|-- lab2-staging (83da75e)
    |-- feature/5-sprint-specification (#51 -> PR #58)
    |-- feature/6-requester-context (#52 -> PR #59)
    |-- feature/7-create-ticket (#53 -> PR #60)
    |-- feature/8-my-tickets (#54 -> PR #61)
    |-- feature/9-ticket-detail (#55 -> PR #62)
    |-- feature/10-zen-green (#56 -> PR #63)
    |-- feature/11-e2e-evidence (#57 -> PR #64)
    |-- feature/12-report-compliance (#65, active report branch)
```

Issue	Feature branch	PR	Current state
#51	feature/ 5-sprint-specification	#58	Merged into lab2-staging by MadMax168.
#52	feature/6-requester-context	#59	Merged into lab2-staging by MadMax168.
#53	feature/7-create-ticket	#60	Merged into lab2-staging by MadMax168.
#54	feature/8-my-tickets	#61	Merged into lab2-staging by MadMax168.
#55	feature/9-ticket-detail	#62	Merged into lab2-staging by MadMax168.
#56	feature/10-zen-green	#63	Merged into lab2-staging by MadMax168.
#57	feature/11-e2e-evidence	#64	Merged into lab2-staging by MadMax168.

The complete review index and response summaries are in [docs/lab-02/reviewer.md](#). The record identifies MadMax168 as the human reviewer and records the requested changes and author responses for PRs #59, #61, #62, and #64. GitHub currently exposes human ready-to-merge or approve comments, but its formal `reviewDecision` is empty for PRs #58-#64; the report must not label those comments as formal **APPROVED** reviews. The project-board state also needs a human capture; the current CLI token cannot read the board without the `read:project` scope.

Required Part 1 captures

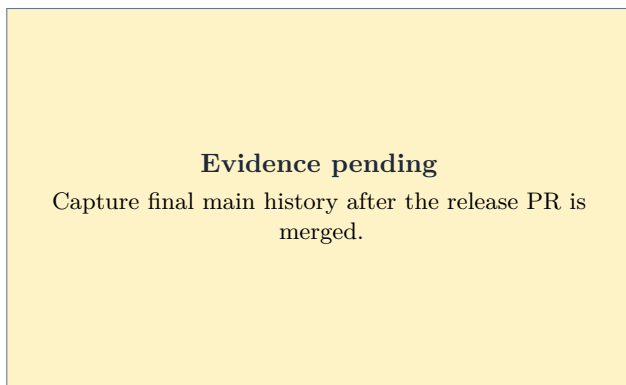


Figure 1: Final `main` history showing feature branches, staging merges, and the release merge.

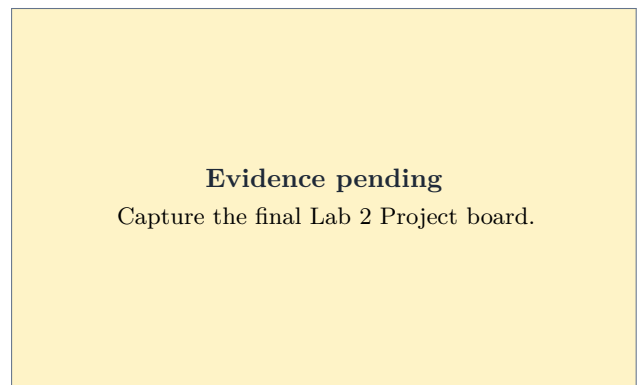


Figure 2: Final Kanban with the Lab 2 Issues in Done.

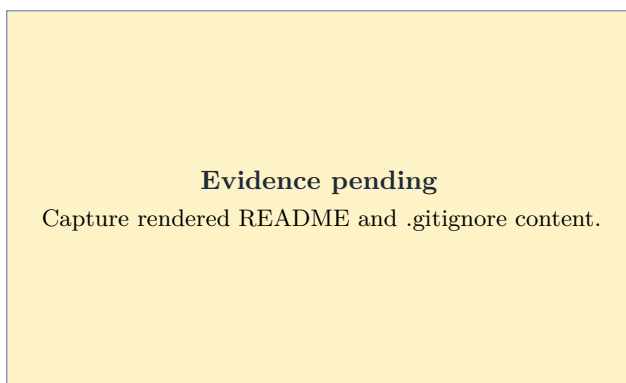


Figure 3: README and `.gitignore` evidence.

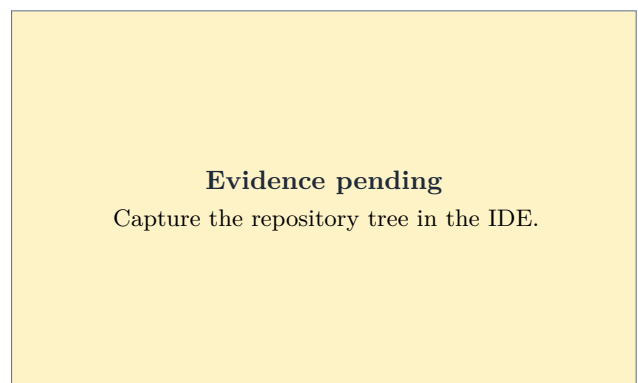


Figure 4: Repository directory structure.

2 Answer Part 2: Spec DD

The complete contract is linked at [docs/lab-02/specification.md](#) and is cross-referenced by the API, UI, and test contracts. It contains 19 functional requirements, 26 business rules, 24 acceptance criteria, and a Definition of Done. The contract-first document was delivered in PR #58 before the implementation PRs were completed.

GitHub records the contract commit as [8f372bd](#) (2026-08-24 06:30 UTC). PR #58 was opened at 06:31 UTC and merged into `lab2-staging` at 09:14 UTC; the next implementation PR, PR #59, was opened later that day. This is the available timestamp evidence for the contract-before-implementation sequence; the final report still needs a readable screenshot of the source and PR timeline.

Contract area	Selected decision	Traceability source
Requester context	Development Requester selection is a test context, not authentication.	FR-01–FR-04, BR-03–BR-07, AC-01–AC-03
Ticket creation	The server owns Ticket Number, Ticket Date, requester ownership, and initial <code>NEW</code> status.	FR-06–FR-08, BR-01–BR-13, AC-05–AC-10
Ownership	List, detail, and attachment operations are scoped to the selected Requester.	FR-09–FR-16, BR-23, AC-11–AC-21
Scope boundary	Authentication, IT Staff workflow, collaboration, and later status transitions remain excluded.	Scope and DoD sections

Rendered contract evidence

The source contract is complete and implemented on `lab2-staging`. The final report still needs a readable rendered copy and one screenshot proving that the specification existed before the main implementation PRs were completed.

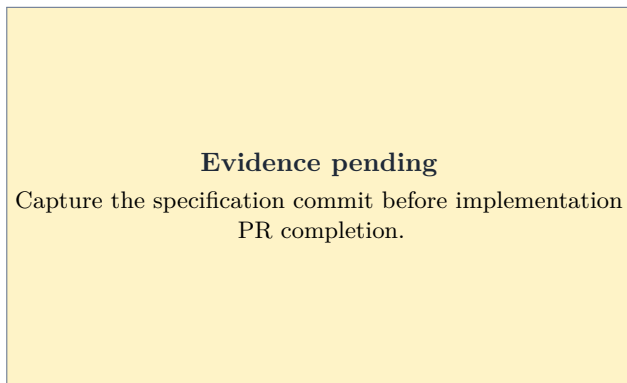


Figure 5: Specification-before-implementation proof, associated with the contract PR.

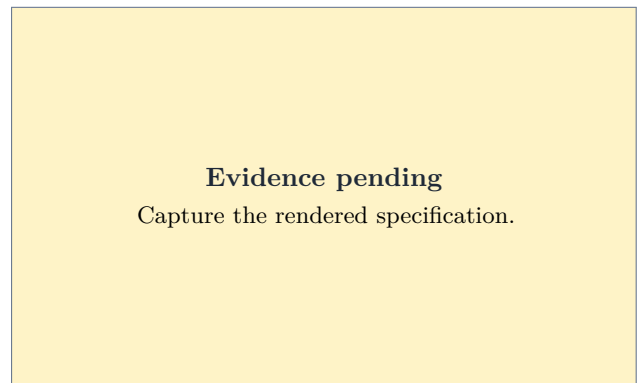


Figure 6: Rendered `specification.md` with numbered FR, BR, AC, and DoD sections.

3 Answer Part 3: Test DD and Traceability

The planned matrix and acceptance-criteria mapping are maintained in [docs/lab-02/tests.md](#). The current integrated staging validation record contains the following results:

Boundary	Result	Main evidence path
Client unit/UI suite	8 files, 32 tests passed	<code>client/tests/lab-01/</code> and <code>client/tests/lab-02/</code>
Server unit/API suite	13 files, 70 tests passed	<code>server/tests/lab-01/</code> and <code>server/tests/lab-02/</code>
Combined test command	21 files, 102 tests passed	<code>bun run test</code>
Browser E2E	2 non-skipped tests and 44 screenshots	<code>e2e/lab-02/</code>
Full repository gate	Passed on the integrated staging baseline	<code>bun run verify</code> ; final-main rerun pending

The principal Lab 2 test paths are:

- `server/tests/lab-02/create-ticket.api.test.ts`, `server/tests/lab-02/my-tickets.api.test.ts`, `server/tests/lab-02/ticket-detail.api.test.ts`, and `server/tests/lab-02/attachments.api.test.ts`;

- `client/tests/lab-02/CreateTicket.test.tsx`, `client/tests/lab-02/MyTickets.test.tsx`, `client/tests/lab-02/RequesterTicketDetail.test.tsx`, and `client/tests/lab-02/AttachmentSection.test.tsx`.

Every AC is mapped to at least one test ID in the Markdown matrix.

The original lifecycle test produced 36 responsive screenshots. The dedicated report-evidence test at `e2e/lab-02/report-evidence.spec.ts` produced eight additional desktop captures for requester selection, Create Ticket states, and the combined My Tickets filter/sort/page state.

The 2026-09-04 elevated local run of `bun run verify` passed on the report branch above the `lab2-staging` product baseline. It checked Biome, Lefthook, Prisma, client and server type checking, 21 test files with 102 tests, and both workspace builds. The normal sandbox invocation stopped at Prisma dependency access with an OS `EPERM`; this is recorded as an environment limitation, not as a passing test result. The labsheet requires complete unit, API, and UI output from final `main`; that evidence cannot be claimed until the release is merged and rerun.

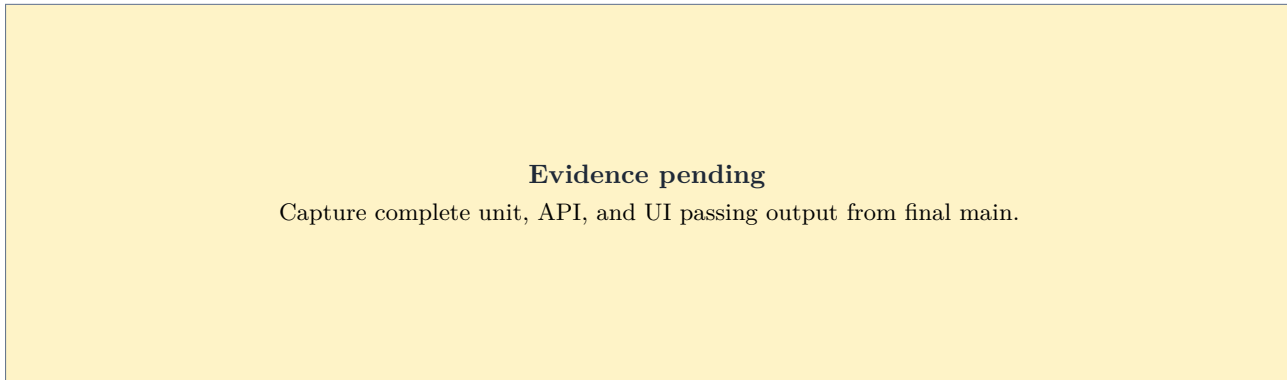


Figure 7: Final-main test output required by the labsheet.

4 Answer Part 4: AI Use with Reflection

The AI-use record is linked at [docs/lab-02/ai-use.md](#). The implementation assistant was OpenAI Codex using GPT-5. The student reviewed the PDF-derived contracts, generated changes, tests, screenshots, and review responses.

ID	Selected prompt focus	Effect
P1	Read the Lab 2 PDF and convert workflows and submission parts into a contract.	Established numbered FR, BR, AC, and DoD traceability.
P2	Create the Issue and branch plan from the contract.	Kept one feature branch per Lab 2 Issue and preserved the staging flow.
P3	Implement data, seed, requester context, API, and ownership boundaries.	Added the requester-facing backend contract without authentication or later-lab scope.
P4	Implement Create Ticket, My Tickets, Detail, and attachments.	Connected the four product areas to the approved API and UI decisions.
P5	Add tests with explicit FR/BR/AC traceability.	Produced unit, API, client UI, responsive, and E2E coverage.
P6	Inspect the requester flow at desktop, tablet, and mobile sizes.	Generated the responsive screenshot inventory and overflow assertions.
P7	Review the complete Lab 2 PR sequence and address comments.	Updated implementation, tests, evidence, and reviewer records after human feedback.
P8	Check report readiness against Answer Parts 1-9.	Separated staging evidence from final-main and missing-capture work.

My Reflection

AI assistance was most useful for translating the labsheet into small, traceable contracts and checking the relationships between implementation, tests, evidence, and review records. Human responsibility remained essential: the student confirmed scope, reviewed generated changes, inspected screenshots, validated the environment, and did not treat automated review output as peer approval.

5 Answer Part 5: Development Requester Select Screen

Lab 2 uses a simulated login screen to select an active Development Requester for the session. It is explicitly not authentication. The repeat-safe seed contains active Requesters A-D and one inactive Requester; the selector returns only the four active records. Continue is disabled until a valid active selection is made, and Change Requester clears the previous requester-specific view.

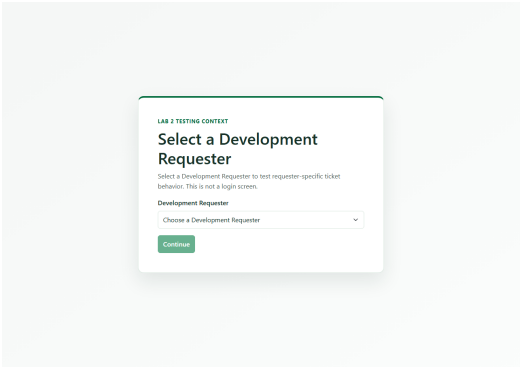


Figure 8: Active requester options with Continue disabled and inactive Requesters excluded.

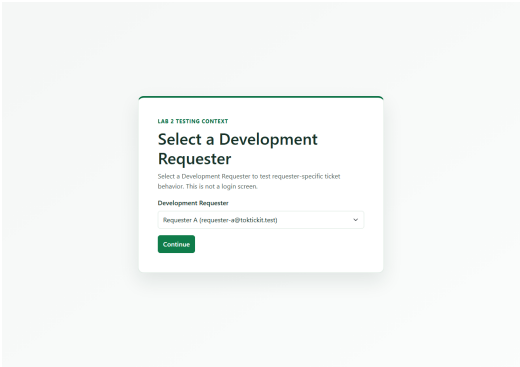


Figure 9: Requester A selected and Continue enabled.

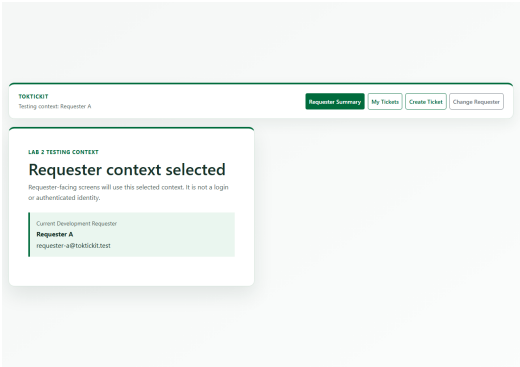


Figure 10: Selected Requester A shown in the requester summary.

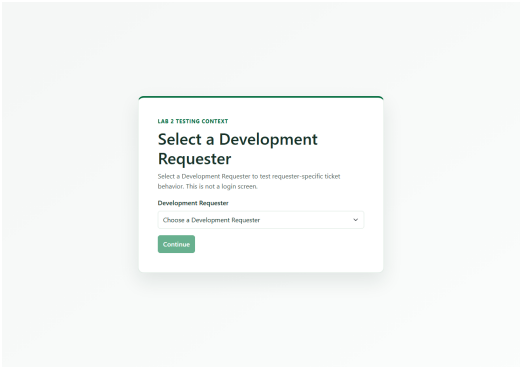


Figure 11: Change Requester returns to a cleared selection screen.

The four captures were collected by the dedicated Playwright evidence test using the seeded active requester shape. They show the selection, enabled continuation, selected-user summary, and reset behavior without presenting this simulated context as authentication.

6 Answer Part 6: Working Ticket Screen - Create Mode

Create Ticket accepts Category, Related System, Requested Priority, Summary, Description, and permitted attachments. The selected requester is displayed as read-only context. The server generates the official Ticket Number, Ticket Date, requester ownership, and initial **NEW** status. Attachment upload is a separate operation so a saved Ticket remains visible if an upload fails.

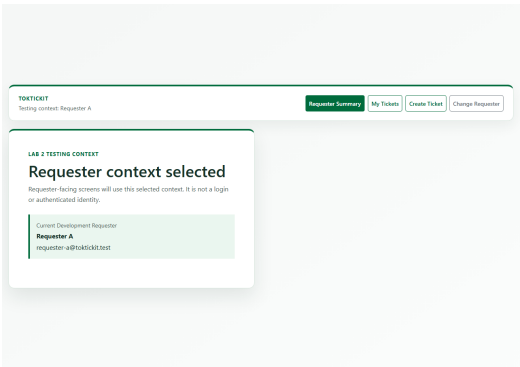


Figure 12: Requester selection and matching context.

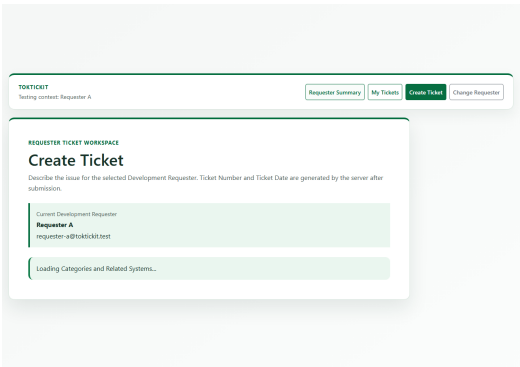


Figure 13: Reference data loading from the API.

The screenshot shows the 'Create Ticket' form in the 'REQUESTER TICKETS' section. The form includes fields for 'Ticket Number' and 'Ticket Date', both marked as required with red asterisks. Below these are dropdown menus for 'Category', 'Related System', and 'Requested Priority', each also marked as required. There are text input fields for 'Summary' and 'Description'. At the bottom, there are checkboxes for 'Attachments' and a 'Submit' button. The form is styled with a light green header and a white body.

Figure 14: Initial editable form with required indicators.

This screenshot shows the 'Create Ticket' form after an invalid submission. Red error messages are displayed below the 'Category', 'Related System', and 'Requested Priority' dropdowns, indicating that they are required fields. The 'Submit' button is disabled, and the form is otherwise identical to Figure 14.

Figure 15: Field-level messages for an invalid submission.

The screenshot shows the 'Create Ticket' form with the 'Submit' button disabled. A red error message is displayed below the 'Description' field, stating 'This ticket represents the same generated ticket result.' The form is otherwise identical to Figure 14.

Figure 16: Disabled Submitting button and duplicate-submit protection.

This screenshot shows the 'Create Ticket' form after a successful submission. The 'Ticket Number' and 'Ticket Date' fields are now populated with values generated by the backend. The 'Submit' button is disabled, and the form is otherwise identical to Figure 14.

Figure 17: Backend-generated Ticket Number and saved values.

The screenshot shows the 'Create Ticket' form after a safe API failure. The 'Submit' button is disabled, and a red error message is displayed below the 'Description' field, stating 'This ticket represents the same generated ticket result.' The form is otherwise identical to Figure 14.

Figure 18: Safe API failure with entered values preserved.

This screenshot shows the 'Create Ticket' form with a file upload section. It displays a list of files, including a permitted PDF file and a rejected file. The 'Submit' button is disabled, and the form is otherwise identical to Figure 14.

Figure 19: One permitted PDF and one rejected file are shown before submission.

The current UI/API tests and E2E evidence support the following contract claims:

- API-02 and E2E-01 verify a successful create response, saved Ticket, backend Ticket Number, and requester context.
- UI-04 and API-03 verify field-level validation and no invalid Ticket persistence.
- UI-05 and API-05 verify busy-submit protection, preserved values, and separate attachment failure behavior.
- The create API test verifies response `requester.id` and then checks the saved database `requesterId`, `categoryId`, `relatedSystemId`, and `currentStatus=NEW`; the response intentionally does not expose a raw `requesterId` field.

7 Answer Part 7: Working My Tickets Screen

My Tickets is always scoped by the selected Development Requester. It supports search over Ticket Number and Summary, Category/Related System/Priority/Status filters, deterministic sorting, one-based pagination, empty and no-results states, and recoverable API failure.

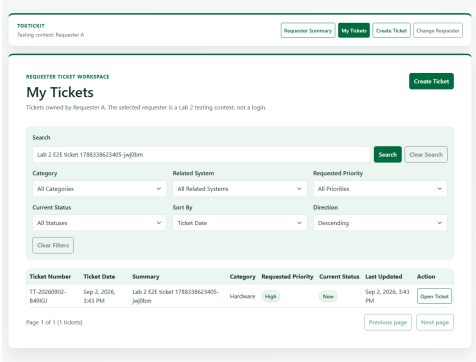


Figure 20: Requester-owned results with list controls.

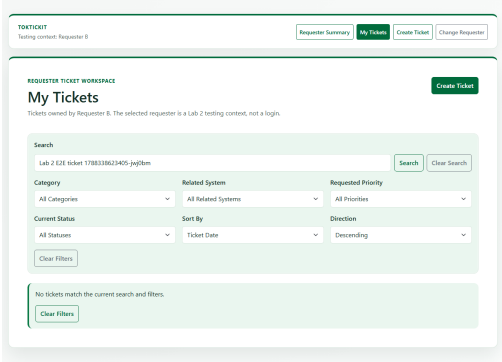


Figure 21: A different Requester cannot see the original Requester's Tickets.

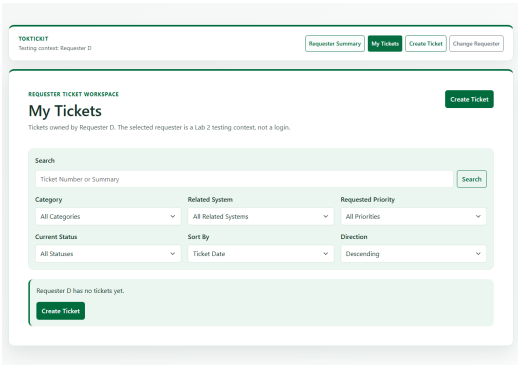


Figure 22: No Tickets for the current Requester.

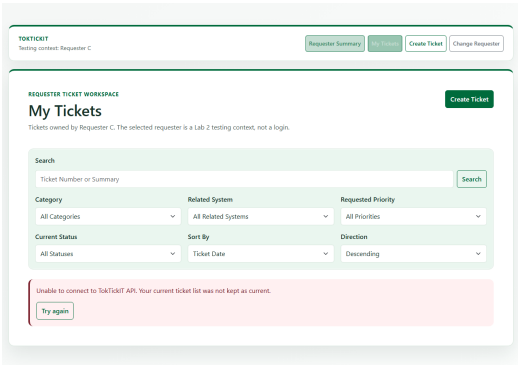


Figure 23: Recoverable list failure.

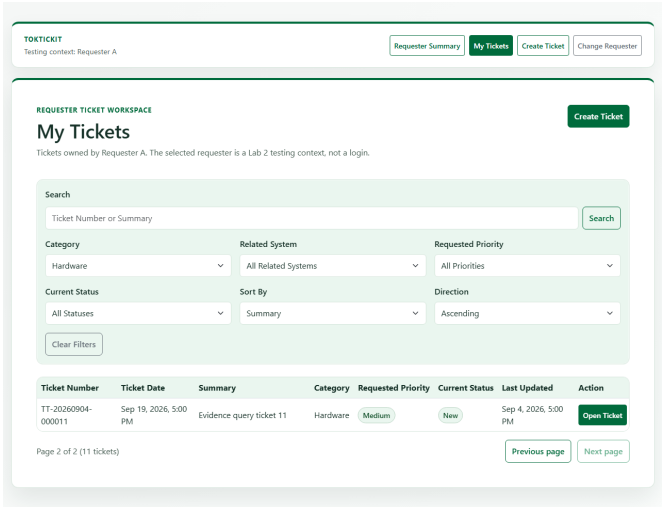


Figure 24: My Tickets with a non-default filter, ascending sort, and page 2 of 2.

The client and server tests verify search, filters, sorting, pagination, ownership isolation, empty/no-results distinction, stale-response protection, and API failure. The additional report capture makes the non-default Category, Summary sort, ascending direction, and page 2 transition visible together. It is a controlled UI-state fixture for the query controls; final-main rerun evidence is still required before submission.

8 Answer Part 8: Working Ticket Screen - View Mode and Attachments

Owned Ticket Detail renders Ticket fields read-only and separates the Attachment section. Active attachments can be uploaded and downloaded. Removal requires confirmation and a 3-200 character reason, retains metadata, and returns 410 ATTACHMENT_REMOVED when a removed file is requested. Cross-Requester Ticket and Attachment access is rejected safely.

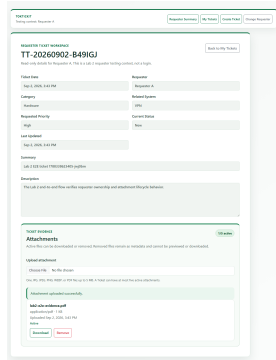


Figure 25: Read-only Ticket Detail with active attachment.

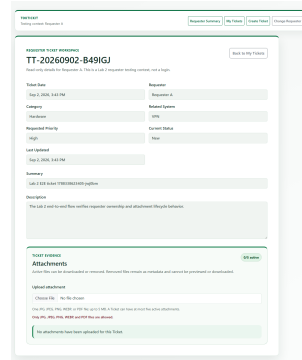


Figure 26: Unsupported attachment feedback.

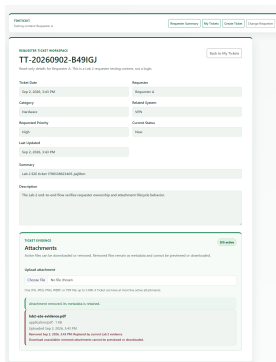


Figure 27: Retained removed metadata and reason.

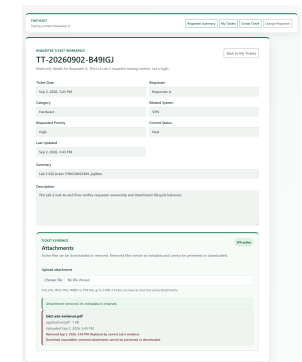


Figure 28: Removed content is not downloadable or pre-viewable.

API-09 through API-16 and E2E-02 cover owned detail, attachment metadata, upload, active download, soft removal, repeat removal, invalid files, storage errors, removed-download rejection, and cross-owner access. The final report should include a readable test-output or API excerpt for the unauthorized Ticket/Attachment requests in addition to the UI screenshots. The relevant test assertions are summarized here:

```
GET /api/tickets/<owned-ticket-id> [Requester B] -> 404 TICKET_NOT_FOUND
GET /api/tickets/<owned-ticket-id>/attachments [Requester B] -> 404 TICKET_NOT_FOUND
POST /api/tickets/<owned-ticket-id>/attachments [Requester B] -> 404 TICKET_NOT_FOUND
GET /api/tickets/<owned-ticket-id>/attachments/<id>/download [Requester B] -> 404 TICKET_NOT_FOUND
DELETE /api/tickets/<owned-ticket-id>/attachments/<id> [Requester B] -> 404 TICKET_NOT_FOUND
```

These are asserted by `server/tests/lab-02/ticket-detail.api.test.ts` and `server/tests/lab-02/attachments.api.test.ts`; no Ticket or Attachment summary is disclosed to the other Requester.

9 Answer Part 9: Zen Green UI and Responsive Evidence

The complete visual contract is linked at [docs/lab-02/ui-spec.md](#). The integrated E2E flow checks 1280x900, 820x900, and 390x844 viewports. The current inventory contains 36 responsive lifecycle captures plus eight desktop report-state captures.

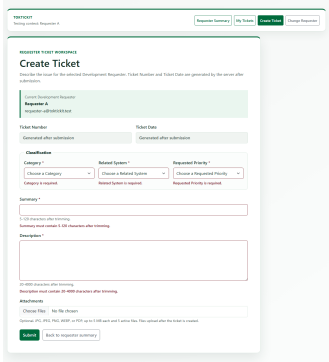


Figure 29: Desktop validation.

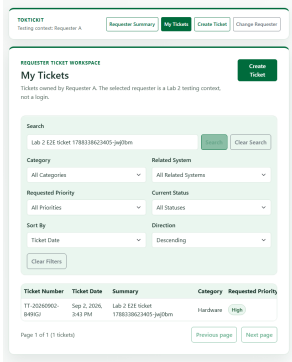


Figure 30: Tablet list controls.



Figure 31: Mobile blocked-download state.

Visual checklist item	Integrated evidence	Final report status
Zen Green colors and shared components	Client contract tests and screenshots	Rendered <code>ui-spec.md</code> and human sign-off pending
Editable/read-only fields	Create and Detail UI tests	Representative screenshots available
Validation placement and state text	Client tests and validation captures	Representative evidence available
Button hierarchy and disabled busy state	Client tests and UI contract	Dedicated submitting capture available
Clipping, overlap, and horizontal overflow	E2E viewport assertions	Final-main rerun and visual sign-off pending

The final report must include the completed visual checklist and all readable desktop, tablet, and mobile evidence. The current screenshot set is a strong integrated staging baseline, but it is not yet the complete final-main submission package.